

数字电路设计

第 7 章—ARM 体系结构与汇编

Digital Design and Computer Architecture (DDCA) Ch6

ARM® Edition —Sarah L. Harris & David Money Harris

考试重点学习笔记

考试重要性: [极高]

任胜兵老师数字电路设计课程

2026 年 6 月

目录

1	ARM 体系结构概述	3
1.1	ARM 简介	3
1.2	四大设计原则 (Hennessy & Patterson)	3
2	ARM 寄存器集 (考试重点)	4
2.1	16 个通用寄存器 (ARM v7, 32 位)	4
2.2	CPSR (当前程序状态寄存器)	4
3	ARM 汇编指令集	5
3.1	数据传输指令	5
3.2	逻辑指令	5
3.3	移位指令	5
3.4	乘法指令	6
3.5	CMP 比较指令	6
4	15 种条件码与条件执行 (考试核心)	7
4.1	NZCV 标志位与条件码对照	7
4.2	条件执行示例	7
4.3	有符号 vs 无符号比较速查	7
5	四种寻址方式 (考试重点 [极高])	8
5.1	寻址方式对比总表	8
5.2	1. 寄存器寻址 (Register Addressing)	8
5.3	2. 立即数寻址 (Immediate Addressing)	8
5.4	3. 基址寻址 (Base Addressing)	9
5.5	三种索引模式 (Indexing Modes)	9
5.6	4. PC 相对寻址 (PC-Relative Addressing)	9
6	三种指令格式编码 (考试重点 [极高])	11
6.1	总览: 通过 op 字段区分	11
6.2	1. 数据处理指令格式 (DP, op=00)	11
6.3	2. 内存指令格式 (Memory, op=01)	12
6.4	3. 分支指令格式 (Branch, op=10)	13
6.5	机器码解读流程	14
7	大小端 (Endianness)	15
7.1	Big-Endian vs Little-Endian	15
8	栈与函数调用	16
8.1	栈 (Stack)	16
8.2	函数调用约定	16

8.3 if-else 语句汇编实现	17
9 字节编址内存	18
10 存储程序概念 (Stored Program)	19
11 考试重点提示 (Exam Tips)	20
11.1 必考知识点清单	20
11.2 常见考试题型	20
11.3 关键公式速查	21
11.4 易错提醒	21

ARM 体系结构概述

ARM 简介

- ARM (Advanced RISC Machines) 由 ARM Holdings 开发于 1980 年代。
- 每年出货近 100 亿颗 ARM 处理器；超过 75% 的人类使用含 ARM 处理器的产品。
- 广泛应用于手机、平板、服务器、相机、机器人、汽车等。
- ARM v7 是 32 位体系结构，每条指令 32 位，每个寄存器 32 位。
- ARM 属于 RISC (Reduced Instruction Set Computer)，指令少而简单；与 Intel x86 (CISC) 形成对比。

四大设计原则 (Hennessy & Patterson)

原则 1: Regularity supports design simplicity (规整性支持设计简单性)

- 一致的指令格式，相同数量的操作数（两个源操作数 + 一个目的操作数）
- 便于硬件编码和处理

原则 2: Make the common case fast (使常见情况快速)

- ARM 只包含简单、常用的指令
- 硬件解码和执行保持简单、小巧、快速
- 复杂操作由多条简单指令组合完成

原则 3: Smaller is faster (越小越快)

- ARM 只有少量寄存器（16 个通用寄存器）
- 寄存器比内存快得多

原则 4: Good design demands good compromises (好的设计需要好的折中)

- 多种指令格式允许灵活性（数据处理/内存/分支三种格式）
- 但格式种类保持最少以遵守原则 1 和 3

ARM 寄存器集（考试重点）

16 个通用寄存器（ARM v7, 32 位）

表 1: ARM v7 寄存器集（32 位体系结构）

寄存器	别名	用途
R0	—	参数 / 返回值 / 临时变量
R1-R3	—	参数 / 临时变量
R4-R11	—	保存变量（被调用者保存）
R12	—	临时变量（调用者保存）
R13	SP (Stack Pointer)	栈指针，指向栈顶
R14	LR (Link Register)	链接寄存器，保存函数返回地址
R15	PC (Program Counter)	程序计数器，指向当前指令 +8

【考点：必须熟记 R13=SP, R14=LR, R15=PC 及其功能】

CPSR（当前程序状态寄存器）

CPSR (Current Program Status Register) 包含条件标志位：

表 2: CPSR 条件标志位（NZCV — 考试必考）

标志	名称	含义与设置条件
N	Negative	ALU 结果为负数时置 1 (bit 31 = 1)
Z	Zero	ALU 结果为零时置 1
C	Carry	ALU 操作产生无符号进位/借位时置 1
V	oVerflow	ALU 操作产生有符号溢出时置 1

设置 NZCV 的两种方法：

方法 1: **CMP** 指令：执行 $R_n - R_m$ ，仅设置标志位，不保存结果

例：CMP R5, R6 （执行 $R5 - R6$ ，只影响 NZCV）

方法 2: 加 **S** 后缀：在指令助记符后加 **s**，计算结果并设置标志位

例：ADDS R1, R2, R3 （执行 $R2 + R3$ ，结果存 R1，同时设置 NZCV）

CMP 等价于 **SUBS** 但不保存结果。

【考点：CMP vs SUBS 的区别是常考考点：CMP 不保存结果只设标志位！】

ARM 汇编指令集

数据传输指令

表 3: 核心数据传输指令

指令	助记符	格式	说明
加法	ADD	ADD Rd, Rn, Op2	$Rd = Rn + Op2$
减法	SUB	SUB Rd, Rn, Op2	$Rd = Rn - Op2$
传送	MOV	MOV Rd, Op2	$Rd = Op2$
载入	LDR	LDR Rd, [Rn, offset]	$Rd = mem[Rn + offset]$
存储	STR	STR Rs, [Rn, offset]	$mem[Rn + offset] = Rs$

示例：

C 代码	ARM 汇编	说明
a = b + c;	ADD R0, R1, R2	$R0 = R1 + R2$
a = b - c;	SUB R0, R1, R2	$R0 = R1 - R2$
a = a + 4;	ADD R0, R0, #4	$R0 = R0 + 4$ (立即数)
b = a - 12;	SUB R1, R0, #12	$R1 = R0 - 12$
int a = 23;	MOV R0, #23	$R0 = 23$
int b = 0x45;	MOV R1, #0x45	$R1 = 0x45$

MOV 立即数限制：立即数必须有 < 8 位精度（通过 8-bit imm8 + 4-bit 旋转编码 32 位常量）。

逻辑指令

表 4: 逻辑运算指令

指令	操作	用途
AND Rd, Rn, Op2	$Rd = Rn \& Op2$	位屏蔽 (保留指定位)
ORR Rd, Rn, Op2	$Rd = Rn Op2$	位合并 (组合位域)
EOR Rd, Rn, Op2	$Rd = Rn \wedge Op2$	异或
BIC Rd, Rn, Op2	$Rd = Rn \& \sim Op2$	位清除 (AND NOT)
MVN Rd, Op2	$Rd = \sim Op2$	取反传送 (Move Not)

掩码示例：

- 保留最低字节：AND R0, R0, #0xFF → 0xF234012F & 0xFF = 0x2F
- 清除最低字节：BIC R0, R0, #0xFF → 0xF234012F & ~0xFF = 0xF2340100
- 合并位域：ORR R0, R1, R2 → 0xF2340000 | 0x12BC = 0xF23412BC

移位指令

- 立即移位量：5-bit immediate (0–31)

表 5: 移位指令（编码 sh 字段）

指令	名称	sh 编码	示例
LSL	逻辑左移	00	LSL R0, R7, #5 → R0 = R7 << 5
LSR	逻辑右移	01	LSR R3, R2, #31 → R3 = R2 >> 31
ASR	算术右移	10	ASR R9, R11, #4 → R9 = R11 >>> 4
ROR	循环右移	11	ROR R8, R1, #3 → R8 = R1 ROR 3

- 寄存器移位量：低 8 位 (0-255)

乘法指令

指令	说明
MUL Rd, Rm, Rs	32 位乘法: $Rd = Rm \times Rs$
MLA Rd, Rm, Rs, Rn	乘加: $Rd = Rm \times Rs + Rn$
UMULL RdHi, RdLo, Rm, Rs	无符号 64 位乘法: $\{RdHi, RdLo\} = Rm \times Rs$
SMULL RdHi, RdLo, Rm, Rs	有符号 64 位乘法

CMP 比较指令

CMP Rn, Op2 执行 $Rn - Op2$ ，仅设置 NZCV 标志位，结果丢弃。

【考点：CMP + 条件后缀 = 条件执行，是 ARM 编程核心模式！】

15 种条件码与条件执行（考试核心）

NZCV 标志位与条件码对照

下表是考试必须完整记忆的 15 种条件码（cond 4-bit 编码）：

表 6: ARM 条件码完整表（15 种 + 1 保留）			
cond[3:0]	助记符	全称	条件表达式（NZCV 测试）
0000	EQ	Equal	Z = 1（相等/为零）
0001	NE	Not Equal	Z = 0（不等）
0010	CS/HS	Carry Set / Higher or Same	C = 1（无符号 ≥）
0011	CC/LO	Carry Clear / Lower	C = 0（无符号 <）
0100	MI	Minus / Negative	N = 1（负数）
0101	PL	Plus / Positive or Zero	N = 0（正数或零）
0110	VS	oVerflow Set	V = 1（溢出）
0111	VC	oVerflow Clear	V = 0（无溢出）
1000	HI	Unsigned Higher	C = 1 且 Z = 0（无符号 >）
1001	LS	Unsigned Lower or Same	C = 0 或 Z = 1（无符号 ≤）
1010	GE	Signed Greater or Equal	N = V（有符号 ≥）
1011	LT	Signed Less Than	N ≠ V（有符号 <）
1100	GT	Signed Greater Than	Z = 0 且 N = V（有符号 >）
1101	LE	Signed Less or Equal	Z = 1 或 N ≠ V（有符号 ≤）
1110	AL	Always (无条件)	总是执行（默认）
1111	（保留）	—	未使用/扩展

【考点：必考！尤其是符号条件 (GE/LT/GT/LE) 对应的 NZCV 测试表达式！】

条件执行示例

条件码附加在指令助记符后，构成条件指令：

```
CMP  R5, R9           ; 执行 R5-R9，设置NZCV
SUBEQ R1, R2, R3       ; 如果 R5==R9 (Z=1)，则执行 SUB
ORRMI R4, R0, R9       ; 如果结果为负 (N=1)，则执行 ORR
```

```
; 例：R5=17, R9=23
; CMP: 17-23=-6 → N=1, Z=0, C=0, V=0
; SUBEQ 不执行 (Z=0)
; ORRMI 执行 (N=1)
```

有符号 vs 无符号比较速查

【考点：有符号比较使用 N 和 V 联合判断 (GT/GE/LT/LE) 是高频考点！】

比较类型	使用的条件码	条件表达式
无符号 >	HI	C=1 且 Z=0
无符号 ≥	CS/HS	C=1
无符号 <	CC/LO	C=0
无符号 ≤	LS	C=0 或 Z=1
有符号 >	GT	Z=0 且 N=V
有符号 ≥	GE	N=V
有符号 <	LT	N≠V
有符号 ≤	LE	Z=1 或 N≠V
相等	EQ	Z=1
不等	NE	Z=0

四种寻址方式（考试重点 [极高]）

任老师原话：”连题目都告诉你了——ARM 指令格式分析、寻址方式识别与转换”。

寻址方式对比总表

表 7: 四种寻址方式完整对比

寻址方式	操作数来源	指令格式	典型 ARM 指令示例
1. 寄存器寻址	寄存器中的值	数据处理格式 (I=0)	ADD R0, R2, R7
2. 立即数寻址	指令中编码的常数	数据处理格式 (I=1)	ADD R9, R1, #14
3. 基址寻址	内存 [基址 + 偏移]	内存格式 (op=01)	LDR R0, [R8, #-11]
4. PC 相对寻址	PC + 有符号偏移	分支格式 (op=10)	B TARGET / BL FUNC

1. 寄存器寻址 (Register Addressing)

操作数位于寄存器中，用于数据处理指令。三种子模式：

子模式	说明	示例
Register-only	直接使用寄存器值	ADD R0, R2, R7
Immediate-shifted register	寄存器值先移位再使用	ORR R5, R1, R3, LSL #1
Register-shifted register	移位量来自另一个寄存器	SUB R12, R9, R0, ASR R1

2. 立即数寻址 (Immediate Addressing)

源操作数是编码在指令中的常数。使用数据处理格式，I=1。

- 编码方式：8-bit imm8 + 4-bit rot（旋转量）
- 32-bit 立即数 = imm8 ROR (rot × 2)
- 例：ADD R9, R1, #14 —14 编码为 imm8=14, rot=0→14 ROR 0 = 14

表 8: 立即数编码示例

rot	32-bit 常量 (imm8 = abcdefgh)
0000	0000 0000 0000 0000 0000 0000 abcd efgh
0001	gh00 0000 0000 0000 0000 0000 00ab cdef
...	...
1111	0000 0000 0000 0000 0000 00ab cdef gh00

注意: 并非所有 32 位常数都能编码为一个立即数, 只有 8 位宽 (带偶旋转) 的值可以直接编码。大常数需要用 MOV + ORR 组合构造。

3. 基址寻址 (Base Addressing)

内存操作数地址 = 基址寄存器 + 偏移。偏移可以是:

偏移类型	示例	地址
12-bit 立即数偏移	LDR R0, [R8, #-11]	R8 - 11
寄存器偏移	LDR R1, [R7, R9]	R7 + R9
立即数移位寄存器偏移	STR R5, [R3, R2, LSL #4]	R3 + (R2 << 4)
零偏移	LDR R0, [R9]	R9

三种索引模式 (Indexing Modes)

基址寻址下的三种索引模式 (考试常考):

表 9: 三种索引模式对比

模式	地址	基址寄存器更新	示例
Offset (偏移)	基址 ± 偏移	不变	LDR R1, [R2, #4]
Pre-index (前索引)	基址 ± 偏移	更新为基址 ± 偏移	LDR R3, [R5, #16]!
Post-index (后索引)	基址	更新为基址 ± 偏移	LDR R8, [R1], #8

- Offset: LDR R1, [R2, #4] → R1 = mem[R2+4], R2 不变
- Pre-index (!): LDR R3, [R5, #16]! → R3 = mem[R5+16], R5 = R5+16
- Post-index: LDR R8, [R1], #8 → R8 = mem[R1], R1 = R1+8

【考点: Pre-index 写回 (!) 和 Post-index 的区别是必考判断!】

4. PC 相对寻址 (PC-Relative Addressing)

用于分支指令 B 和 BL。

- BTA (Branch Target Address) = (PC+8) + SignExtended(imm24) × 4
- imm24: 有符号 24 位立即数, 表示目标地址距离 PC+8 的指令字数

- 注意：PC 在 ARM 流水线中指向当前指令 +8 的位置（三级流水线）

示例：

```
0xA0  BLT THERE      ; PC=0xA0, PC+8=0xA8
0xA4  ADD R0,R1,R2
0xA8  SUB R0,R0,R9
0xAC  ADD SP,SP,#8
0xB0  MOV PC,LR
0xB4  THERE SUB R0,R0,#1 ; BTA=0xB4
; imm24 = (0xB4 - 0xA8) / 4 = 3
; 机器码：0xBA000003
```

反向跳转示例：

```
0x8050 BL TEST      ; PC=0x8050, PC+8=0x8058
; TEST at 0x8040
; imm24 = (0x8040 - 0x8058) / 4 = -6（有符号）
; 机器码：0xEBFFFFFFA
```

【考点：BTA 计算公式必须掌握： $(PC+8) + imm24 \times 4$ ！注意 imm24 是有符号数。】

三种指令格式编码（考试重点 [极高]）

总览：通过 op 字段区分

表 10: 三种指令格式—op 字段总览

op[27:26]	类型	编码指令
00	Data-processing (DP)	ADD, SUB, MOV, AND, ORR, EOR, BIC, MVN, CMP, LSL, LSR, ASR,
01	Memory	LDR, STR, LDRB, STRB
10	Branch	B, BL

1. 数据处理指令格式 (DP, op=00)

完整 32 位字段布局（寄存器 Src2, I=0, bit4=0）:

31:28	27:26	25	24:21	20	19:16	15:12	11:7	6:5	4	3:0
cond	op=00	I=0	cmd	S	Rn	Rd	shamt5	sh	0	Rm

立即数 Src2 (I=1):

31:28	27:26	25	24:21	20	19:16	15:12	11:8	7:0
cond	op=00	I=1	cmd	S	Rn	Rd	rot	imm8

各字段详解:

- **cond [31:28]**: 条件码 (4-bit), 1110 = AL (无条件)
- **op [27:26]**: 操作类型, 00 = 数据处理
- **I [25]**: 0 = Src2 为寄存器, 1 = Src2 为立即数
- **cmd [24:21]**: 具体操作码 (4-bit)
- **S [20]**: 1 = 设置条件标志位 (S 后缀)
- **Rn [19:16]**: 第一源寄存器
- **Rd [15:12]**: 目的寄存器
- **shamt5 [11:7]**: 移位量 (仅 I=0 时)
- **sh [6:5]**: 移位类型码 (仅 I=0 时)
- **bit4 [4]**: 0 = 寄存器, 1 = 寄存器移位寄存器
- **Rm [3:0]**: 第二源寄存器

cmd 操作码 (4-bit):

【考点: cmd 编码是”给题目直接分析机器码”的核心! 必须会解码。】

编码示例:

例 1: ADD R0, R1, #42 → 0xE281002A

cmd	指令	说明	cmd	指令	说明
0000	AND	逻辑与	1000	TST	测试 (AND, 不存结果)
0001	EOR	异或	1001	TEQ	测试 (EOR, 不存结果)
0010	SUB	减法	1010	CMP	比较 (SUB, 不存结果)
0011	RSB	反向减法	1011	CMN	比较加 (ADD, 不存结果)
0100	ADD	加法	1100	ORR	逻辑或
0101	ADC	带进位加	1101	(shift)	移位指令
0110	SBC	带借位减	1110	BIC	位清除
0111	RSC	带借位反向减	1111	MVN	取反传送

- cond=1110, op=00, I=1, cmd=0100(ADD), S=0, Rn=1(R1), Rd=0(R0)
- rot=0000, imm8=00101010(=42)

例 2: SUB R2, R3, #0xFF0 → 0xE2432EFF

- cond=1110, op=00, I=1, cmd=0010(SUB), S=0, Rn=3, Rd=2
- imm8=0xFF, rot=14 (ROR by 28 to get 0xFF0)

例 3: ADD R5, R6, R7 → 0xE0865007

- cond=1110, op=00, I=0, cmd=0100(ADD), S=0, Rn=6(R6), Rd=5(R5)
- Rm=7(R7), shamt5=0, sh=00

例 4: ORR R9, R5, R3, LSR #2 → 0xE1859123

- cond=1110, op=00, I=0, cmd=1100(ORR), S=0, Rn=5, Rd=9
- Rm=3, shamt5=2, sh=01(LSR)

例 5: ROR R1, R2, #23 → 0xE1A01BE2

- cond=1110, op=00, I=0, cmd=1101(shift), S=0, Rn=0, Rd=1
- Rm=2, shamt5=23(=10111), sh=11(ROR)

【考点：从机器码反解汇编指令也是常见题型！】

2. 内存指令格式 (Memory, op=01)

完整 32 位字段布局：

31:28	27:26	25	24	23	22	21	20	19:16	15:12	11:0
cond	op=01	I	P	U	B	W	L	Rn	Rd	Src2 (offset)

funct 字段详解 (6 个控制位 [25:20]):

L 和 B 组合决定指令类型:

P 和 W 组合决定索引模式:

编码示例:

例 1: STR R11, [R5], #-26 → 0xE405B01A

表 11: Memory 格式 funct 控制位

位	名称	含义
I [25]	Immediate bar	0= 立即数偏移, 1= 寄存器偏移
P [24]	Pre-index	0= 后索引, 1= 前索引/偏移
U [23]	Up/Add	0= 减去偏移, 1= 加上偏移
B [22]	Byte	0= 字 (32-bit), 1= 字节 (8-bit)
W [21]	Writeback	1= 写回基址 (配合 P 使用)
L [20]	Load	0=STR(存储), 1=LDR(载入)

L	B	指令
0	0	STR (存储字)
0	1	STRB (存储字节)
1	0	LDR (载入字)
1	1	LDRB (载入字节)

P	W	索引模式
0	0	Post-index (后索引)
0	1	不支持
1	0	Offset (偏移)
1	1	Pre-index (前索引, 写回)

- cond=1110, op=01
- I=0(imm), P=0(post), U=0(sub), B=0(word), W=0(post), L=0(store)
- Rd=11(R11), Rn=5(R5), imm12=26

例 2: LDR R3, [R4, R5] → 0xE7943005

- cond=1110, op=01
- I=1(reg), P=1(offset), U=1(add), B=0(word), W=0(offset), L=1(load)
- Rd=3, Rn=4, Rm=5, shamt5=0, sh=0

例 3: STR R9, [R1, R3, LSL #2] → 0xE7819103

- cond=1110, op=01
- I=1(reg), P=1(offset), U=1(add), B=0(word), W=0(offset), L=0(store)
- Rd=9, Rn=1, Rm=3, shamt5=2, sh=00(LSL)

3. 分支指令格式 (Branch, op=10)

完整 32 位字段布局:

31:28	27:26	25:24	24	23:0
cond	op=10	保留	L	imm24

- **cond [31:28]**: 条件码
- **op [27:26]**: 10 = 分支指令
- **L [24]**: 0 = B (分支), 1 = BL (分支并链接)
- **imm24 [23:0]**: 有符号 24 位立即数, 表示相对 PC+8 的偏移指令字数 (**words**)

BTA 计算公式:

$$BTA = (PC + 8) + \text{SignExtend}(\text{imm24}) \times 4$$

【考点: imm24 是有符号数, 乘以 4 是因为每条指令 4 字节! 这是考试陷阱点。】

机器码解读流程

通用机器码反解步骤:

1. 看 op[27:26] 确定指令类型: 00=DP, 01=Memory, 10=Branch
2. 若 DP (op=00): 看 I[25] 确定 Src2 类型; 若 I=0, 看 bit4 确定是否寄存器移位寄存器
3. 若 Memory (op=01): 看 funct[I,P,U,B,W,L] 确定索引模式和指令
4. 若 Branch (op=10): 看 L[24] 确定 B 还是 BL; imm24 计算 BTA

示例 (机器码 → 汇编):

- 0xE0475001: op=00(DP), I=0, cmd=0010(SUB), Rn=7, Rd=5, Rm=1, shamt5=0, sh=0 → SUB R5, R7, R1
- 0xE5949010: op=01(Mem), funct: L=1(LDR), B=0, P=1(offset), I=0(imm), U=1(add), Rn=4, Rd=9, imm12=16 → LDR R9, [R4, #16]

【考点: 给你一个 32 位十六进制机器码, 要能分析出是什么指令!】

大小端 (Endianness)

Big-Endian vs Little-Endian

- **Little-Endian** (小端): 最低有效字节 (LSB) 存储在最低地址
- **Big-Endian** (大端): 最高有效字节 (MSB) 存储在最低地址
- ARM 默认支持小端模式

示例: 假设 $R5 = 0x23456789$, $R2 = 8$ (内存地址)

执行 `STR R5, [R2, #0]` 后:

模式	Address 8	Address 9	Address A	Address B
Big-Endian	0x23 (MSB)	0x45	0x67	0x89 (LSB)
Little-Endian	0x89 (LSB)	0x67	0x45	0x23 (MSB)

若再执行 `LDRB R7, [R2, #1]`:

- Big-endian: $R7 = 0x00000045$
- Little-endian: $R7 = 0x00000067$

【考点: 给定内存布局能判断是大端还是小端!】

栈与函数调用

栈 (Stack)

- 栈用于临时保存变量（如函数调用时保存寄存器）
- LIFO (Last-In-First-Out) 数据结构
- 向低地址增长（从高地址向低地址扩展）
- SP (R13) 始终指向栈顶
- ARM 使用 Full Descending 栈模式

栈操作（没有专用 PUSH/POP 指令，使用 STR/LDR 模拟）：

```
; 压栈（保存3个寄存器）
SUB  SP, SP, #12    ; 分配空间
STR  R4, [SP, #8]   ; 保存 R4
STR  R8, [SP, #4]   ; 保存 R8
STR  R9, [SP]       ; 保存 R9
```

```
; 弹栈（恢复3个寄存器）
LDR  R9, [SP]       ; 恢复 R9
LDR  R8, [SP, #4]   ; 恢复 R8
LDR  R4, [SP, #8]   ; 恢复 R4
ADD  SP, SP, #12    ; 释放空间
```

函数调用约定

表 12: ARM 函数调用约定	
项目	约定
调用指令	BL label（Branch and Link：跳转并保存返回地址到 LR）
返回指令	MOV PC, LR（将 LR 中保存的返回地址写入 PC）
参数传递	R0–R3（前 4 个参数放入 R0-R3）
返回值	R0（函数返回结果放入 R0）
被调用者保存寄存器	R4–R11（函数内部如使用需先保存到栈再恢复）
调用者保存寄存器	R0–R3, R12（调用者如需保留需自行保存）

完整函数调用示例：

```
; 调用者（main）
MOV  R0, #2        ; 参数0 = 2
MOV  R1, #3        ; 参数1 = 3
MOV  R2, #4        ; 参数2 = 4
```

```
MOV R3, #5      ; 参数3 = 5
BL  DIFFOFSUMS  ; 调用函数 (LR = 下一条指令地址)
MOV R4, R0      ; y = 返回值
```

; 被调用者 (diffofsums) — 含栈保存

```
DIFFOFSUMS:
SUB SP, SP, #12 ; 分配栈空间
STR R4, [SP, #8] ; 保存 R4
STR R8, [SP, #4] ; 保存 R8
STR R9, [SP]     ; 保存 R9
ADD R8, R0, R1   ; R8 = f + g
ADD R9, R2, R3   ; R9 = h + i
SUB R4, R8, R9   ; result = (f+g) - (h+i)
MOV R0, R4      ; 返回值放入 R0
LDR R9, [SP]     ; 恢复 R9
LDR R8, [SP, #4] ; 恢复 R8
LDR R4, [SP, #8] ; 恢复 R4
ADD SP, SP, #12 ; 释放栈空间
MOV PC, LR      ; 返回
```

【考点：BL 执行时，LR = PC+4（下一条指令地址）；返回用 MOV PC, LR。】

if-else 语句汇编实现

表 13: if 语句的两种 ARM 汇编实现方式

C 代码	方式 1: 条件分支	方式 2: 条件执行
if (i == j) f = g + h; f = f - i;	CMP R3, R4	CMP R3, R4
	BNE L1	
	ADD R0,R1,R2	ADDEQ R0,R1,R2
	L1	
	SUB R0,R0,R2	SUB R0,R0,R2

方式 2 利用 ARM 条件执行特性，省去分支跳转，代码更紧凑。

字节编址内存

- 每个字节有唯一地址
- 32 位字 = 4 字节，因此字地址以 4 递增
- 内存字地址 = 字序号 $\times$ 4
 - 字 2 的地址 = $2 \times 4 = 8$
 - 字 10 的地址 = $10 \times 4 = 40 = 0x28$
- LDR/STR 操作 32 位字；LDRB/STRB 操作 8 位字节

存储程序概念 (Stored Program)

- 32 位指令和数据存储在内存中
- 不同应用程序的区别仅在于内存中的指令序列
- 运行新程序：只需在内存中存储新程序，无需重新布线
- PC 跟踪当前指令地址，处理器按序从内存取指执行

考试重点提示 (Exam Tips)

必考知识点清单

1. ARM 寄存器全图: R0-R15 (SP=R13, LR=R14, PC=R15), 熟记用途
2. CPSR 四标志位: N (Negative), Z (Zero), C (Carry), V (oVerflow) — 熟记设置条件
3. 15 种条件码完整表: EQ/NE/CS/CC/MI/PL/VS/VC/HI/LS/GE/LT/GT/LE/AL — 熟记 cond 编码和 NZCV 测试表达式
4. 条件标志位设置方法: CMP (不存结果) vs S 后缀 (存结果 + 设标志)
5. 四种寻址方式识别: 寄存器/立即数/基址偏移/PC 相对 — 给出指令能判断
6. 三种索引模式: Offset / Pre-index(!) / Post-index — 会区分
7. 三种指令格式编码: op=00(DP), op=01(Memory), op=10(Branch) — 会解析 32 位机器码
8. BTA 计算公式: $(PC+8) + \text{SignExtend}(\text{imm24}) \times 4$ — 注意 PC+8 偏移
9. 大小端判断: 给定内存布局, 能判断 Big-Endian 还是 Little-Endian
10. 函数调用: BL(跳转保存 LR), MOV PC, LR(返回), R0-R3 传参, R0 返回值
11. 栈操作: 向低地址增长, 用 STR/LDR+SP 模拟 PUSH/POP
12. 立即数编码: imm8 ROR (rot×2) 机制

常见考试题型

- 题型 1: 指令格式分析: 给定 ARM 汇编指令, 写出 32 位机器码 (或反向), 标出各字段含义
- 题型 2: 寻址方式识别: 给定指令, 判断属于哪种寻址方式 (寄存器/立即数/基址/PC 相对)
- 题型 3: 条件执行分析: 给定 CMP 和条件指令序列, 判断哪些指令会执行
- 题型 4: 机器码解码: 给定 0x 开头 32 位十六进制数, 分析出对应的 ARM 汇编指令
- 题型 5: BTA 计算: 给定 PC 值和分支指令, 计算目标地址
- 题型 6: 大小端分析: 给定内存布局, 判断大小端模式
- 题型 7: C 代码 → 汇编: 将简单 if/else、循环、函数调用翻译为 ARM 汇编
- 题型 8: 寄存器追踪: 跟踪程序执行过程中各寄存器的值变化

项目	公式/规则
BTA (分支目标地址)	$BTA = (PC+8) + \text{SignExtend}(\text{imm24}) \times 4$
imm24	$\text{imm24} = (BTA - (PC+8)) / 4$ (有符号整数)
32 位立即数	imm8 ROR (rot $\times 2$)
字地址	字地址 = 字序号 $\times 4$
BL 链接地址	LR = PC + 4 (下一条指令地址)
栈增长方向	向低地址增长 (高 $\rightarrow$ 低)

关键公式速查

易错提醒

- PC = 当前指令地址 + 8, 不是 +4 (三级流水线: 取指/译码/执行)
- CMP 不保存结果, 只设置标志位
- 有符号比较用 GE/LT/GT/LE (涉及 N 和 V), 无符号用 HS/LO/HI/LS (涉及 C 和 Z)
- imm24 是指令字数偏移, 乘以 4 得到字节偏移
- 条件码 1110 是 AL (无条件), 不是 1111
- 寄存器寻址有三种子模式: register-only, immediate-shifted, register-shifted

祝考试顺利!